Netspy-World + Refunds.site Staging Integration

Quick Setup Guide for Replit Team

Environment: STAGING

Status: Ready for Testing

Date: June 28, 2026

---

1. Quick Start (5 Minutes)

Step 1: Request Decryption Key

Plain Text

```
Email: integration-security@netspyworld.com
Subject: Request Decryption Key - Staging Integration
Body:
  Team: Replit
  Environment: Staging
  Use Case: Netspy-Refunds Integration
  Key ID: mk_netspy_staging_001
```

Response: PGP-encrypted email with decryption key (within 1 hour)

Step 2: Decrypt Credentials

Using Node.js:

Bash

```
# Install dependencies
npm install crypto

# Create decrypt.js
cat > decrypt.js << 'EOF'
import crypto from 'crypto';

const encryptedData = '7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c
const iv = 'a1b2c3d4e5f6g7h8i9j0k1l2';
const authTag = 'm3n4o5p6q7r8s9t0u1v2w3x4';
```

```
const key = process.env.DECRYPTION_KEY; // From PGP email

const decipher = crypto.createDecipheriv('aes-256-gcm', Buffer.from(key, 'hex')
decipher.setAuthTag(Buffer.from(authTag, 'hex'));

let decrypted = decipher.update(encryptedData, 'hex', 'utf8');
decrypted += decipher.final('utf8');

console.log('Decrypted API Key:', decrypted);
EOF

# Run with decryption key
DECRYPTION_KEY=<key_from_email> node decrypt.js
```

Using Python:

Bash

```
# Install dependencies
pip install cryptography

# Create decrypt.py
cat > decrypt.py << 'EOF'
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
import binascii

encrypted_data = '7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7
iv = 'a1b2c3d4e5f6g7h8i9j0k1l2'
auth_tag = 'm3n4o5p6q7r8s9t0u1v2w3x4'
key = os.getenv('DECRYPTION_KEY')  # From PGP email

cipher = AESGCM(binascii.unhexlify(key))
plaintext = cipher.decrypt(
    binascii.unhexlify(iv),
    binascii.unhexlify(encrypted_data + auth_tag),
    None
)

print(f'Decrypted API Key: {plaintext.decode()}')
EOF

# Run with decryption key
DECRYPTION_KEY=<key_from_email> python decrypt.py
```

Step 3: Store in Replit Secrets

1. Go to Replit project
2. Click "Secrets" (lock icon)
3. Add each credential:

Plain Text

```
NETSPY_API_KEY=sk_staging_netspy_refunds_a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6
NETSPY_API_SECRET=sk_secret_staging_refunds_x9y8z7w6v5u4t3s2r1q0p9o8n7m6l5k4
NETSPY_WEBHOOK_SECRET=whk_secret_staging_refunds_z1y2x3w4v5u6t7s8r9q0p1o2n3m4l5
NETSPY_STAGING_URL=https://staging-api.netspyworld.com/api/trpc
```

Step 4: Test Connection

Bash

```
# Create test.js
cat > test.js << 'EOF'
import axios from 'axios';

async function testConnection( ) {
  try {
    const response = await axios.get(
      'https://staging-api.netspyworld.com/api/health',
      {
        headers: {
          'Authorization': `Bearer ${process.env.NETSPY_API_KEY}`,
          'X-API-Key': process.env.NETSPY_API_KEY
        }
      }
     );

    console.log('✅ Connection successful!');
    console.log('Status:', response.data.status);
    console.log('Services:', response.data.services);
  } catch (error) {
    console.error('❌ Connection failed:', error.message);
  }
}

testConnection();
EOF
```

```
# Run test
node test.js
```

Expected Output:

Plain Text

```
✅ Connection successful!
Status: healthy
Services: {
  database: 'healthy',
  cache: 'healthy',
  blockchain: 'healthy',
  webhooks: 'healthy'
}
```

---

2. API Integration

Create Unified Case

TypeScript

```typescript
import axios from 'axios';
import crypto from 'crypto';

async function createUnifiedCase() {
  const apiKey = process.env.NETSPY_API_KEY;
  const apiSecret = process.env.NETSPY_API_SECRET;

  const caseData = {
    clientEmail: 'test@staging.refunds.site',
    clientName: 'Test Client',
    caseType: 'both',
    estimatedLoss: 5000,
    caseDescription: 'Staging test case for integration validation'
  };

  // Create signature
  const payload = JSON.stringify(caseData);
  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(payload)
    .digest('hex');

  try {
```

```
    const response = await axios.post(
      'https://staging-api.netspyworld.com/api/trpc/cases.createUnified',
      caseData,
      {
        headers: {
          'Authorization': `Bearer ${apiKey}`,
          'X-API-Key': apiKey,
          'X-API-Signature': signature,
          'Content-Type': 'application/json'
        }
      }
     );

    console.log('✅ Case created successfully!');
    console.log('Unified Case ID:', response.data.unifiedCaseId);
    console.log('Case Number:', response.data.caseNumber);

    return response.data;
  } catch (error) {
    console.error('❌ Failed to create case:', error.response?.data || error.me
    throw error;
  }
}

// Run
createUnifiedCase();
```

Retrieve Case

TypeScript

```
async function getUnifiedCase(unifiedCaseId: number) {
  const apiKey = process.env.NETSPY_API_KEY;

  try {
    const response = await axios.get(
      `https://staging-api.netspyworld.com/api/trpc/cases.getUnified?unifiedCas
      {
        headers: {
          'Authorization': `Bearer ${apiKey}`,
          'X-API-Key': apiKey,
          'Content-Type': 'application/json'
        }
      }
     );
```

```
    console.log('✅ Case retrieved successfully!');
    console.log('Investigation Status:', response.data.investigation.status);
    console.log('Recovery Status:', response.data.recovery.status);
    console.log('Recovery Percentage:', response.data.unifiedCase.recoveryPerce

    return response.data;
  } catch (error) {
    console.error('❌ Failed to retrieve case:', error.response?.data || error.
    throw error;
  }
}

// Run
getUnifiedCase(1);
```

3. Webhook Setup

Create Webhook Receiver

TypeScript

```
import express from 'express';
import crypto from 'crypto';

const app = express();
app.use(express.json());

// Webhook receiver
app.post('/webhooks/netspy-events', (req, res) => {
  const signature = req.headers['x-webhook-signature'];
  const timestamp = req.headers['x-webhook-timestamp'];
  const deliveryId = req.headers['x-webhook-delivery-id'];
  const payload = JSON.stringify(req.body);

  const secret = process.env.NETSPY_WEBHOOK_SECRET;

  // Verify signature
  const signedContent = `${timestamp}.${payload}`;
  const expectedSignature = crypto
    .createHmac('sha256', secret)
    .update(signedContent)
    .digest('hex');

  if (signature !== `sha256=${expectedSignature}`) {
    console.error('❌ Invalid webhook signature');
```

```
    return res.status(401).json({ error: 'Invalid signature' });
  }

  // Verify timestamp (5 minute window)
  const now = Math.floor(Date.now() / 1000);
  const ts = parseInt(timestamp);
  if (Math.abs(now - ts) > 300) {
    console.error('❌ Webhook timestamp outside acceptable window');
    return res.status(401).json({ error: 'Timestamp outside window' });
  }

  console.log(`✅ Webhook received: ${req.body.event} (Delivery ID: ${deliveryI

  // Handle event
  switch (req.body.event) {
    case 'case.created':
      handleCaseCreated(req.body.data);
      break;
    case 'case.statusUpdated':
      handleCaseStatusUpdated(req.body.data);
      break;
    case 'blockchain.traceCompleted':
      handleBlockchainTraceCompleted(req.body.data);
      break;
    case 'vault.created':
      handleVaultCreated(req.body.data);
      break;
    case 'vault.complianceVerified':
      handleVaultComplianceVerified(req.body.data);
      break;
    case 'vault.funded':
      handleVaultFunded(req.body.data);
      break;
    case 'vault.withdrawalCompleted':
      handleVaultWithdrawalCompleted(req.body.data);
      break;
    default:
      console.warn(`Unknown event type: ${req.body.event}`);
  }

  res.json({ success: true, processed: true });
});

// Event handlers
function handleCaseCreated(data: any) {
  console.log(`📋 Case Created: ${data.unifiedCaseId}`);
  console.log(`   Client: ${data.clientName} (${data.clientEmail})`);
  console.log(`   Loss: $${data.estimatedLoss}`);
```

```
  // TODO: Update Refunds database
}

function handleCaseStatusUpdated(data: any) {
  console.log(`📊 Case Status Updated: ${data.unifiedCaseId}`);
  console.log(`   Investigation: ${data.investigationStatus}`);
  console.log(`   Recovery: ${data.recoveryStatus}`);
  // TODO: Update Refunds dashboard
}

function handleBlockchainTraceCompleted(data: any) {
  console.log(`⛓️  Blockchain Trace Completed: ${data.traceId}`);
  console.log(`   Risk Score: ${data.summary.overallRiskScore}`);
  console.log(`   Recovery Potential: ${data.summary.recoveryPotential}%`);
  // TODO: Update path-of-funds visualization
}

function handleVaultCreated(data: any) {
  console.log(`🔐 Vault Created: ${data.vault.id}`);
  console.log(`   Address: ${data.vault.address}`);
  console.log(`   Chain: ${data.vault.chain}`);
  // TODO: Store vault address in Refunds database
}

function handleVaultComplianceVerified(data: any) {
  console.log(`✅ Vault Compliance Verified: ${data.vaultId}`);
  console.log(`   OFAC: ${data.complianceResult.ofacScreening}`);
  console.log(`   AML Rating: ${data.complianceResult.amlRating}`);
  // TODO: Unlock recovery actions
}

function handleVaultFunded(data: any) {
  console.log(`💰 Vault Funded: ${data.vaultId}`);
  console.log(`   Amount: $${data.amount}`);
  console.log(`   TX Hash: ${data.txHash}`);
  // TODO: Update portfolio balance
}

function handleVaultWithdrawalCompleted(data: any) {
  console.log(`🎉 Withdrawal Completed: ${data.withdrawal.id}`);
  console.log(`   Amount: $${data.withdrawal.amount}`);
  console.log(`   TX Hash: ${data.withdrawal.txHash}`);
  // TODO: Update case status to completed
}

// Start server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
```

```
  console.log(`🚀 Webhook receiver listening on port ${PORT}`);
  console.log(`📍 Webhook URL: https://refunds.site/api/public/refunds/webhooks
});
```

4. Feature Flag Activation

Enable Live Sync

TypeScript

```typescript
// In your Replit feature flag system
const featureFlags = {
  NETSPY_INTEGRATION_ENABLED: true,
  NETSPY_STAGING_MODE: true,
  NETSPY_LIVE_SYNC: true,
  NETSPY_WEBHOOK_RECEIVER: true,
  NETSPY_AUTO_CASE_CREATION: true,
  NETSPY_AUTO_VAULT_CREATION: true
};

// Check flag before operations
if (featureFlags.NETSPY_INTEGRATION_ENABLED) {
  // Enable Netspy integration features
  enableNetsyIntegration();
}
```

5. Testing Checklist

- [ ] Health check endpoint responds (200 OK)
- [ ] Create test case (201 Created)
- [ ] Retrieve test case (200 OK)
- [ ] Webhook receiver accepts POST requests
- [ ] Webhook signature verification works
- [ ] Case status updates sync correctly
- [ ] Blockchain trace data received
- [ ] Vault creation triggers webhook
- [ ] Compliance verification completes

- [ ] Withdrawal completion updates case

6. Monitoring & Debugging

View Logs

Bash

```
# Check webhook delivery logs
curl -X GET https://staging-api.netspyworld.com/api/logs \
  -H "Authorization: Bearer $NETSPY_API_KEY" \
  -H "X-API-Key: $NETSPY_API_KEY"

# Filter by event type
curl -X GET "https://staging-api.netspyworld.com/api/logs?event=case.created" \
  -H "Authorization: Bearer $NETSPY_API_KEY"

# Filter by case ID
curl -X GET "https://staging-api.netspyworld.com/api/logs?caseId=1" \
  -H "Authorization: Bearer $NETSPY_API_KEY"
```

Debug Webhook Delivery

Bash

```
# Get webhook delivery status
curl -X GET https://staging-api.netspyworld.com/api/webhooks/deliveries \
  -H "Authorization: Bearer $NETSPY_API_KEY"

# Retry failed delivery
curl -X POST https://staging-api.netspyworld.com/api/webhooks/deliveries/{deliv
  -H "Authorization: Bearer $NETSPY_API_KEY"
```

7. Troubleshooting

"Invalid API Key" Error

Solution:

1. Verify key is stored correctly in Replit Secrets
2. Check key hasn't expired (90-day rotation)

3. Confirm you're using staging URL (not production)
4. Request new key if expired

"Webhook Signature Invalid" Error

Solution:

1. Verify webhook secret is correct
2. Check timestamp is within 5-minute window
3. Ensure payload hasn't been modified
4. Verify HMAC-SHA256 algorithm is used

"Rate Limit Exceeded" Error

Solution:

1. Implement exponential backoff
2. Check rate limit tier (staging: 1,000 req/hour)
3. Contact support for higher limits
4. Batch requests when possible

"Connection Timeout" Error

Solution:

1. Check staging API status: https://staging-status.netspyworld.com
2. Verify network connectivity
3. Check firewall rules
4. Increase timeout to 30 seconds

---

8. Next Steps

Immediate (Today)

- [ ] Request decryption key
- [ ] Decrypt credentials
- [ ] Store in Replit Secrets
- [ ] Test health check

Short-term (This Week)

- [ ] Create test case
- [ ] Retrieve test case
- [ ] Set up webhook receiver
- [ ] Test webhook delivery

Medium-term (This Month)

- [ ] Load test (100+ concurrent requests)
- [ ] Security audit
- [ ] Penetration testing
- [ ] Activate feature flag for live sync

Long-term (Before Production)

- [ ] Performance optimization
- [ ] Disaster recovery testing
- [ ] Generate production credentials
- [ ] Migrate to production environment

---

9. Support Contacts

Integration Support:

- Email: staging-support@netspyworld.com
- Slack: #netspy-refunds-staging
- Response Time: 1 hour for P1 issues

Security Issues:

- Email: integration-security@netspyworld.com
- PGP Key: [Available on request]
- Response Time: 30 minutes for security issues

Escalations:

- Email: integration-escalations@netspyworld.com
- Response Time: 15 minutes for critical issues

Ready to begin? Start with Step 1 above!

Questions? Contact: staging-support@netspyworld.com